

Kubernetes-rapport – EKS Todo-applikation (IaC)

Namn: Andreas Vilhelmsson

Kurs: Kubernetes

GitHub-repo: <https://github.com/AndreasVilhelmsson/kubernetes-AWS>

Sammanfattning

Den här rapporten beskriver ett komplett, återupprepningsbart Kubernetes-flöde på AWS EKS för en Todo-applikation bestående av **React (frontend)**, **.NET 9 (backend)** och **MongoDB**. Klustret och nätverk skapas med **Terraform (IaC)**, containrar byggs och publiceras med **GitHub Actions** till **GitHub Container Registry (GHCR)**, och deployment till Kubernetes sker med manifest/Helm (med ett dedikerat **namespace** och konsekventa **tags** på resurserna). Fokus har varit **låg kostnad student rapport**, möjlighet att **riva upp och ner** utan manuell påverkan, samt **portabilitet** mellan ARM64 (lokal Mac) och AMD64 (EKS noder) genom multi-arch images. projektet har varit utmanande och suttit i flera dagar för att lösa problem. Detta är mitt tredje projekt som jag har gjort med kubernetes. Har varit extra stora utmaningar för att jag sitter på en mac dator med Arm processor. Första gången jag satt upp allt så gick hela flödet igenom. Beslutade mig dock för att riva det då ARM64 och AMD64 inte kunde köra ihop.

Innehåll

1. [Fundament i ett Kubernetes-kluster](#)
 2. [Vanliga Kubernetes-objekt \(kind:\)](#)
 3. [Administration lokalt och i molnet](#)
 4. [Egen applikation på EKS](#)
 - [Design & arkitektur](#)
 - [IaC: Terraform för VPC + EKS](#)
 - [Containrar & CI/CD \(GitHub Actions till GHCR\)](#)
 - [Kubernetes-manifest/Helm](#)
 - [Säkerhet](#)
 - [Driftsättning & test](#)
 5. [Arkitekturdiagram \(Mermaid\)](#)
 6. [Skärmdumpar](#)
 7. [Lärdomar och fallgropar](#)
 8. [Referenser](#)
-

Fundament i ett Kubernetes-kluster

Kubernetes Deployment - Todo Application

Inlämningsuppgift Cloud Services

Namn: Andreas Vilhelmsson
Kurs: Cloud Services
Git Repository: <https://github.com/andreasvilhelmsson/eks-mongo-todo>
Datum: Oktober 2025

Sammanfattning

Detta projekt demonstrerar deployment av en fullstack todo-applikation till AWS EKS (Elastic Kubernetes Service). Applikationen består av en React/TypeScript frontend, .NET backend API, och MongoDB databas. Projektet omfattar infrastruktur som kod med Terraform, CI/CD med GitHub Actions, och Kubernetes-manifest för orchestrering.

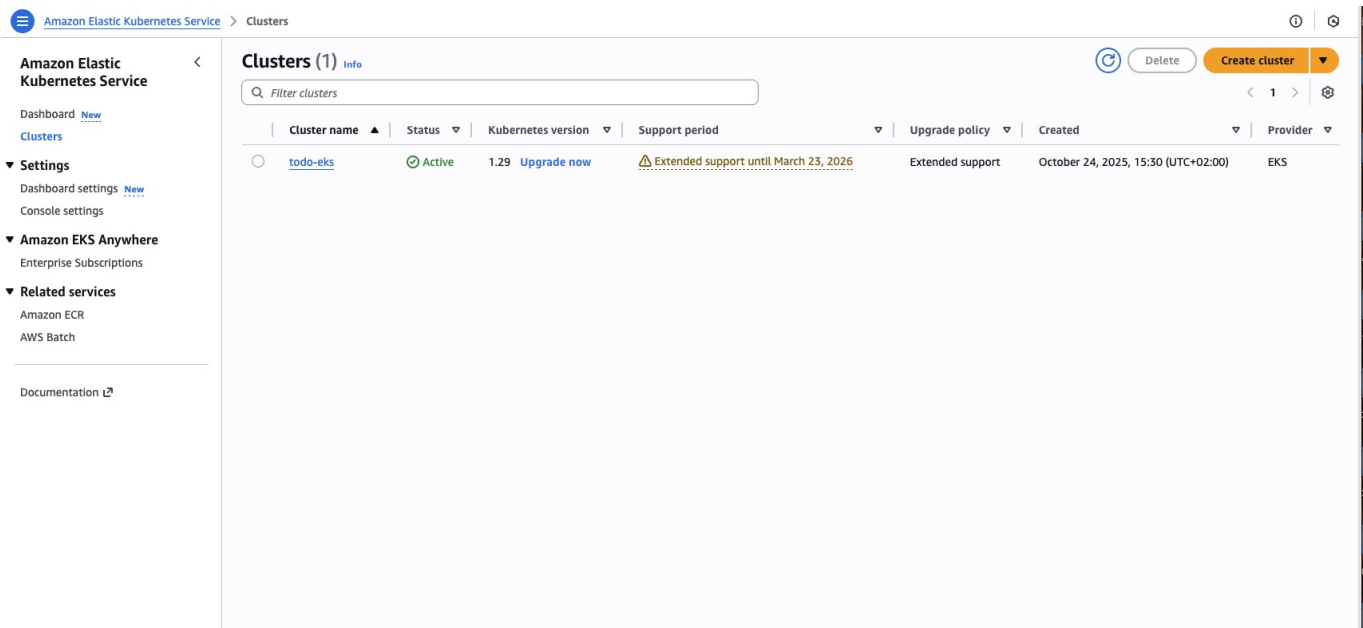
Teknologier:

- Kubernetes (AWS EKS)
- Docker & Container Registry (GitHub Container Registry)
- Terraform (Infrastructure as Code)
- React + TypeScript + SASS (Frontend)
- .NET 9 (Backend API)
- MongoDB (Database)
- NGINX Ingress Controller
- AWS VPC, EBS, NLB

1. Kubernetes Kluster - Fundamentala Komponenter

Ett Kubernetes-kluster består av två huvudsakliga delar: **Control Plane** och **Worker Nodes**.

1.1 Control Plane Komponenter



AWS EKS Console visar klustret `todo-eks` med status, version och endpoint.

API Server (kube-apiserver)

- **Funktion:** Klustrets frontend och centrala kommunikationspunkt
- **Ansvar:**
 - Exponerar Kubernetes API
 - Validerar och processar REST-requests
 - Uppdaterar etcd med kluster-state
- **I praktiken:** Alla kubectl-kommandon kommunicerar med API Server

etcd

- **Funktion:** Distribuerad key-value databas
- **Ansvar:**
 - Lagrar all kluster-konfiguration och state
 - Backup och restore-punkt för klustret
- **I praktiken:** Innehåller alla Deployments, Services, ConfigMaps, etc.

Scheduler (kube-scheduler)

- **Funktion:** Beslutar vilken node en pod ska köras på
- **Ansvar:**
 - Analyserar resource requirements (CPU, minne)
 - Kontrollerar node constraints och affinity rules
 - Tilldelar pods till lämpliga nodes
- **I praktiken:** När du skapar en Deployment, väljer Scheduler vilken node som får köra poden

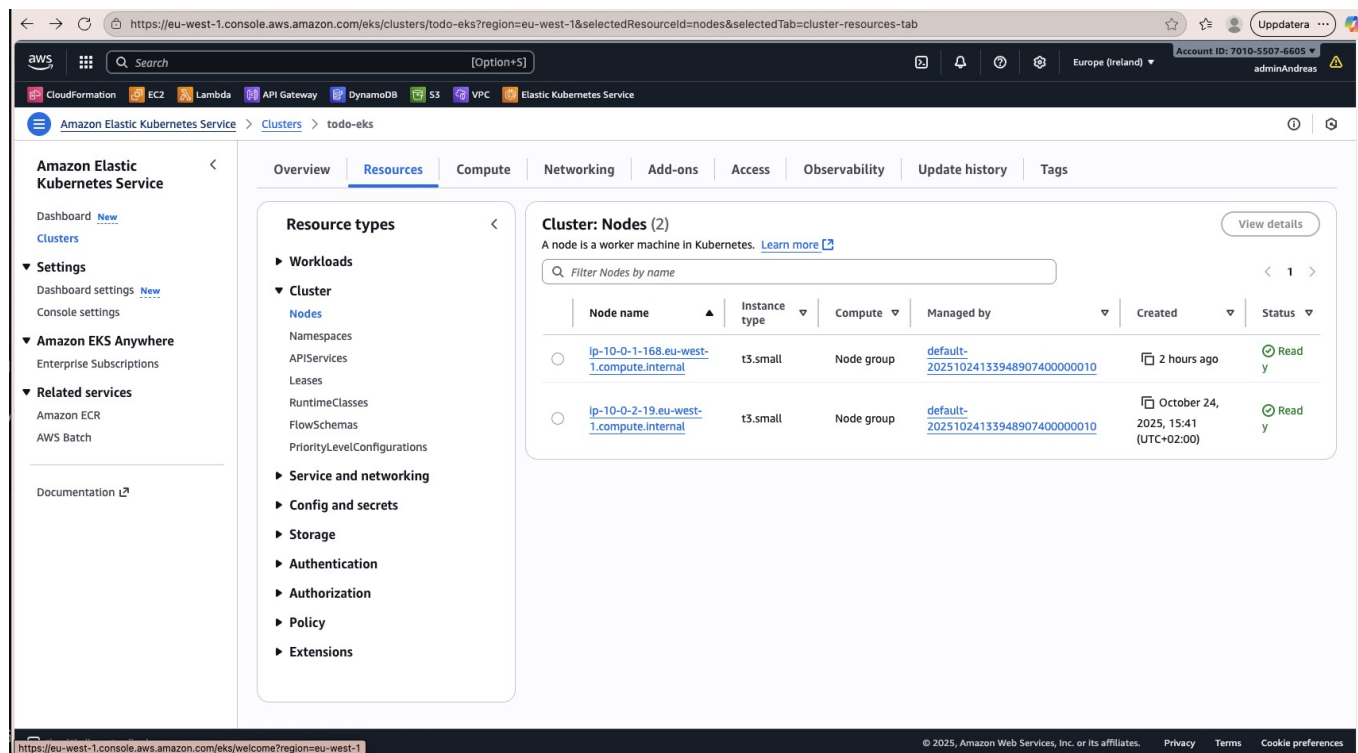
Controller Manager (kube-controller-manager)

- **Funktion:** Kör olika controllers som övervakar kluster-state
- **Ansvar:**
 - Node Controller: Övervakar node-hälsa
 - Replication Controller: Säkerställer rätt antal pod-replicas
 - Endpoints Controller: Populerar Endpoints-objekt
 - Service Account Controller: Skapar default service accounts
- **I praktiken:** Om en pod kraschar, upptäcker Controller Manager detta och skapar en ny

Cloud Controller Manager

- **Funktion:** Integrerar med cloud provider (AWS i vårt fall)
- **Ansvar:**
 - Node Controller: Kontrollerar om nodes har tagits bort i cloud
 - Route Controller: Sätter upp routes i cloud infrastructure
 - Service Controller: Skapar/uppdaterar cloud load balancers
- **I praktiken:** När vi skapar en Service type=LoadBalancer, skapar Cloud Controller Manager en AWS NLB

1.2 Worker Node Komponenter



EC2 Console visar de två t3.small worker nodes och tillhörande taggar.

kubelet

- **Funktion:** Agent som körs på varje node
- **Ansvar:**
 - Tar emot PodSpecs från API Server
 - Säkerställer att containers körs och är healthy
 - Rapporterar node och pod status tillbaka till API Server
- **I praktiken:** Startar och övervakar Docker containers på noden

kube-proxy

- **Funktion:** Nätverksproxy på varje node
- **Ansvar:**
 - Implementerar Kubernetes Service-koncept
 - Hanterar network rules (iptables/IPVS)
 - Möjliggör load balancing mellan pods
- **I praktiken:** När du anropar en Service, routar kube-proxy trafiken till rätt pod

Container Runtime

- **Funktion:** Kör containers
- **Ansvar:**
 - Pullar container images
 - Startar och stoppar containers
 - Hanterar container lifecycle
- **I praktiken:** Docker, containerd, eller CRI-O som faktiskt kör applikationerna

1.3 Add-ons i vårt kluster

CoreDNS

- **Funktion:** DNS-server för klustret
- **Ansvar:** Service discovery - översätter service-namn till IP-adresser
- **I praktiken:** Backend kan nå MongoDB via `mongodb:27017` istället för IP

AWS EBS CSI Driver

- **Funktion:** Container Storage Interface för AWS EBS
- **Ansvar:**
 - Dynamisk provisioning av EBS volumes
 - Attach/detach volumes till nodes
 - Snapshot och restore
 - EBS Amazons molnbaserade hårddiskar – används för att lagra data på ett säkert
- **I praktiken:** MongoDB använder EBS CSI för persistent storage (5Gi gp3 volume)

NGINX Ingress Controller

- **Funktion:** Ingress controller för HTTP/HTTPS routing
- **Ansvar:**
- En Ingress Controller är komponenten i Kubernetes som tar emot trafik utifrån och skickar den vidare till rätt Service/Pod i ditt kluster
- Exponerar Services externt via Load Balancer
- HTTP routing baserat på host/path
- SSL/TLS termination
- **I praktiken:** Routar `/api/*` till backend och `/` till frontend

2. Kubernetes Objekt (kind:) - Typer och Användning

I Kubernetes är kind ett nyckelord i YAML-filer som talar om vilken typ av resurs (objekt) du vill skapa i klustret. Det kan till exempel vara en Pod, Deployment, Service, ConfigMap, Ingress, Job.

2.1 Namespace

Funktion: Logisk isolering av resurser inom klustret

Användning:

```
apiVersion: v1
kind: Namespace
metadata:
  name: eks-mongo-todo
  labels:
    app.kubernetes.io/managed-by: terraform
    project: eks-mongo-todo
```

I vårt projekt:

- Alla applikationsresurser ligger i namespace `eks-mongo-todo`
- Separerar vår app från system-pods (kube-system, ingress-nginx)
- Möjliggör resource quotas och RBAC per namespace

Relation: Namespace är överst i hierarkin - alla andra objekt tillhör ett namespace

2.2 Pod

Funktion: Minsta deployerbara enheten - en eller flera containers som delar nätverk och storage

Användning: Pods skapas vanligtvis inte direkt, utan via Deployments eller StatefulSets.

I vårt projekt:

```
kubectl get pods -n eks-mongo-todo
```

NAME	READY	STATUS
mongodb-0	1/1	Running
todo-backend-55d96ff964-2wj6k	1/1	Running
todo-frontend-5f5b6987d7-rwf5r	1/1	Running

Relation:

- Skapas av Deployment/StatefulSet
 - Exponeras av Service
 - Scheduleras på Node av Scheduler
-

2.3 Deployment

Funktion: Deklarativ hantering av Pods och ReplicaSets - för stateless applikationer

Användning:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: todo-backend
  namespace: eks-mongo-todo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: todo-backend
  template:
    metadata:
      labels:
        app: todo-backend
    spec:
      containers:
```

```
- name: api
  image: ghcr.io/andreasvilhelmsson/todo-backend-v2:latest
  ports:
    - containerPort: 8080
  env:
    - name: MONGO_URI
      value: mongodb://root:changeme@mongodb:27017/?
        authSource=admin
```

Fördelar:

- Rolling updates utan downtime
- Rollback till tidigare version
- Self-healing - återskapar pods vid failure
- Scaling - enkelt öka/minska replicas

I vårt projekt:

- Backend Deployment: 1 replica av .NET API
- Frontend Deployment: 1 replica av React app

Relation:

- Skapar och hanterar ReplicaSet
- ReplicaSet skapar och hanterar Pods
- Service pekar på Pods via labels

2.4 StatefulSet

Funktion: Hantering av stateful applikationer med persistent identity och storage

Användning:

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mongodb
  namespace: eks-mongo-todo
spec:
  serviceName: mongodb
  replicas: 1
  selector:
    matchLabels:
      app: mongodb
  template:
    metadata:
      labels:
        app: mongodb
    spec:
      containers:
```

```
- name: mongo
  image: mongo:7
  ports:
    - containerPort: 27017
  volumeMounts:
    - name: data
      mountPath: /data/db
volumeClaimTemplates:
- metadata:
  name: data
  spec:
    accessModes: ["ReadWriteOnce"]
    storageClassName: ebs-gp3
    resources:
      requests:
        storage: 5Gi
```

Skillnad mot Deployment:

- Stable network identity: `mongodb-0`, `mongodb-1`, etc.
- Persistent storage per pod
- Ordered deployment och scaling
- Ordered rolling updates

I vårt projekt:

- MongoDB StatefulSet med 1 replica
- Persistent volume på 5Gi EBS gp3
- Data överlever pod restarts

Relation:

- Skapar PersistentVolumeClaim per replica
- Service (headless) ger stable DNS namn
- Pods får ordnade namn: `<statefulset-name>-<ordinal>`

2.5 Service

Funktion: Abstraktion som exponerar Pods via ett stabilt nätverk endpoint

Typer:

ClusterIP (default)

Intern IP - endast tillgänglig inom klustret

```
apiVersion: v1
kind: Service
metadata:
  name: todo-backend
```



```
namespace: eks-mongo-todo
spec:
  selector:
    app: todo-backend
  ports:
    - port: 80
      targetPort: 8080
```

I vårt projekt:

- Backend Service: Exponerar backend pods på port 80
- Frontend Service: Exponerar frontend pods på port 80
- MongoDB Service: Headless service för StatefulSet

LoadBalancer

Exponerar Service externt via cloud load balancer

```
apiVersion: v1
kind: Service
metadata:
  name: ingress-nginx-controller
  namespace: ingress-nginx
spec:
  type: LoadBalancer
  selector:
    app: ingress-nginx
  ports:
    - port: 80
      targetPort: 80
```

I vårt projekt:

- NGINX Ingress Controller använder LoadBalancer
- Skapar AWS Network Load Balancer automatiskt

Relation:

- Selector matchar Pod labels
- Endpoints-objekt skapas automatiskt med Pod IPs
- Ingress routar trafik till Services

2.6 Ingress

Funktion: HTTP/HTTPS routing till Services baserat på host och path

Användning:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: todo-ingress
  namespace: eks-mongo-todo
  annotations:
    nginx.ingress.kubernetes.io/proxy-connect-timeout: "600"
spec:
  ingressClassName: nginx
  rules:
    - http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: todo-backend
                port:
                  number: 80
          - path: /
            pathType: Prefix
            backend:
              service:
                name: todo-frontend
                port:
                  number: 80
```

Fördelar:

- En Load Balancer för flera Services
- Path-based routing
- SSL/TLS termination
- URL rewriting

I vårt projekt:

- `/api/*` → backend Service
- `/*` → frontend Service
- Exponeras via AWS NLB

Relation:

- Kräver Ingress Controller (NGINX i vårt fall)
- Routar till Services (inte direkt till Pods)
- Ingress Controller skapar LoadBalancer Service

Vanliga Kubernetes-objekt (kind:)

2.7 ConfigMap och Secret

ConfigMap

Funktion: Lagra konfigurationsdata som key-value pairs

Användning:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  API_URL: "http://backend:80"
  LOG_LEVEL: "info"
```

I vårt projekt:

- Används inte explicit, men environment variables i Deployments fyller samma syfte
- Bättre practice: Flytta MONGO_URI till ConfigMap

Secret

Funktion: Lagra känslig data (lösenord, tokens, nycklar)

Användning:

```
apiVersion: v1
kind: Secret
metadata:
  name: mongo-credentials
type: Opaque
stringData:
  username: root
  password: changeme
```

I vårt projekt:

- MongoDB credentials hårdkodade i Deployment (ej production-ready)
- Bättre practice: Använd Secret och referera via secretKeyRef

Relation:

- Monteras som volumes eller environment variables i Pods
- Base64-encoded (inte krypterad!)
- För verklig säkerhet: AWS Secrets Manager eller HashiCorp Vault

2.8 PersistentVolume (PV) och PersistentVolumeClaim (PVC)

Funktion: Abstraktion för persistent storage

StorageClass

Definierar typer av storage som kan provisioneras dynamiskt

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ebs-gp3
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
  encrypted: "true"
volumeBindingMode: WaitForFirstConsumer
```

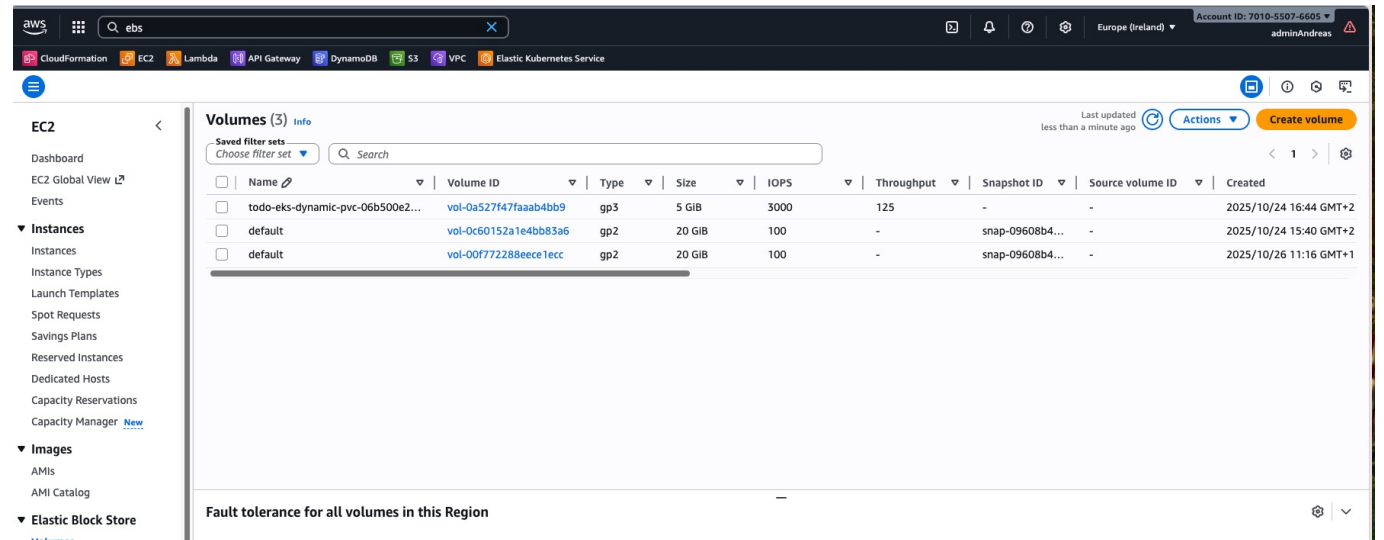
PersistentVolumeClaim

Request för storage från en Pod

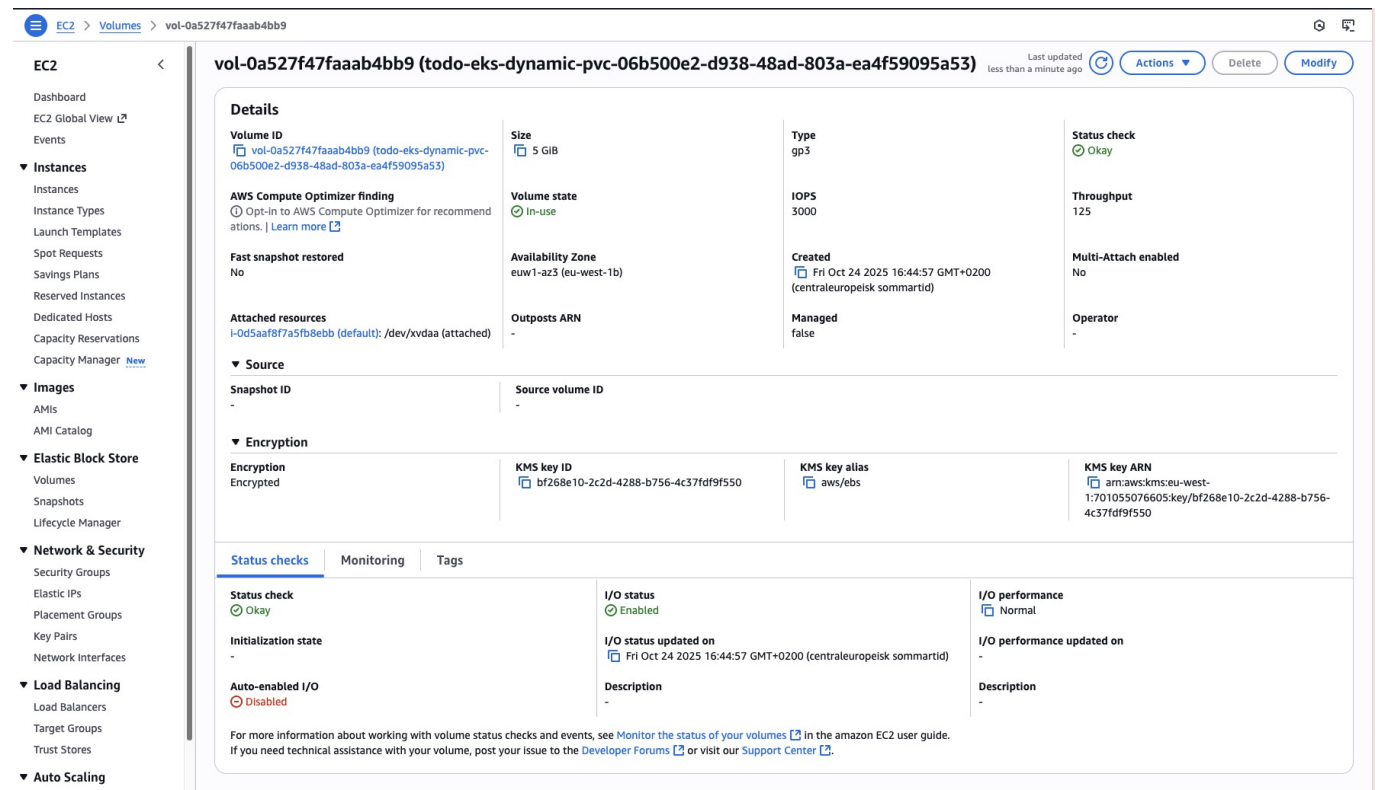
```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mongodb-data
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: ebs-gp3
  resources:
    requests:
      storage: 5Gi
```

I vårt projekt:

- StatefulSet skapar automatiskt PVC via volumeClaimTemplates
- EBS CSI Driver provisionerar EBS volume dynamiskt
- MongoDB data persisteras på EBS gp3 volume



AWS EBS Console visar 5 GiB gp3-volym som används av MongoDB.



Detaljvy med attachment till worker node och taggar från StatefulSet.

Relation:

- StorageClass → PVC → PV → Pod
- CSI Driver hanterar lifecycle
- Volume följer Pod vid reschedule (samma node)

3. Administration av Kubernetes Kluster

3.1 Lokalt Kluster

Minikube

Användning:

```
# Starta lokalt kluster
minikube start --driver=docker --cpus=2 --memory=4096

# Deploya applikation
kubectl apply -f k8s/

# Exponera service lokalt
minikube service todo-frontend --url

# Stoppa kluster
minikube stop
```

Fördelar:

- Snabb utveckling och testning
- Ingen kostnad
- Fungerar på laptop

Nackdelar:

- Begränsade resurser
- Ingen cloud-integration
- Single-node

Kind (Kubernetes in Docker)**Användning:**

```
# Skapa kluster
kind create cluster --name todo-dev

# Ladda lokala images
kind load docker-image todo-backend:latest --name todo-dev

# Ta bort kluster
kind delete cluster --name todo-dev
```

Fördelar:

- Multi-node kluster lokalt
- CI/CD testing
- Snabbare än Minikube

Docker Desktop Kubernetes**Användning:**

- Aktivera i Docker Desktop settings

- Automatisk kubectl konfiguration
 - Enklast för Mac/Windows användare
-

3.2 Molnbaserat Kluster (AWS EKS)

AWS EKS (Elastic Kubernetes Service)

Fördelar:

- Managed Control Plane (AWS hanterar master nodes)
- Integrerat med AWS services (IAM, VPC, EBS, ELB)
- Auto-scaling och auto-healing
- Production-ready

Administration via AWS CLI:

```
# Skapa kluster (manuellt)
aws eks create-cluster \
  --name todo-eks \
  --role-arn arn:aws:iam::123456789:role/eks-cluster-role \
  --resources-vpc-config subnetIds=subnet-xxx,subnet-yyy

# Konfigurera kubectl
aws eks update-kubeconfig --region eu-west-1 --name todo-eks

# Lista kluster
aws eks list-clusters

# Beskriva kluster
aws eks describe-cluster --name todo-eks
```

Administration via Terraform (vårt projekt):

```
module "eks" {
  source = "terraform-aws-modules/eks/aws"
  version = "~> 19.0"

  cluster_name      = "todo-eks"
  cluster_version   = "1.29"

  vpc_id            = module.vpc.vpc_id
  subnet_ids        = module.vpc.public_subnets

  eks_managed_node_groups = {
    default = {
      instance_types = ["t3.small"]
      capacity_type  = "SPOT"

      desired_size = 2
    }
  }
}
```

```
        min_size      = 2
        max_size      = 2
    }
}
```

Deployment workflow:

```
# 1. Skapa infrastruktur
cd infra/terraform
terraform init
terraform apply

# 2. Konfigurera kubectl
aws eks update-kubeconfig --region eu-west-1 --name todo-eks

# 3. Verifiera anslutning
kubectl get nodes

# 4. Deploya applikation
kubectl apply -f k8s/
```

3.3 kubectl - Kommandoradsverktyg

Grundläggande kommandon:

```
# Visa resurser
kubectl get pods -n eks-mongo-todo
kubectl get services -n eks-mongo-todo
kubectl get deployments -n eks-mongo-todo

# Detaljerad information
kubectl describe pod mongodb-0 -n eks-mongo-todo
kubectl describe service todo-backend -n eks-mongo-todo

# Loggar
kubectl logs deployment/todo-backend -n eks-mongo-todo --tail=50
kubectl logs -f pod/mongodb-0 -n eks-mongo-todo # Follow logs

# Exec into pod
kubectl exec -it mongodb-0 -n eks-mongo-todo -- mongosh

# Port forwarding (för lokal testning)
kubectl port-forward svc/todo-backend 8080:80 -n eks-mongo-todo

# Apply manifests
kubectl apply -f k8s/
kubectl apply -f k8s/backend/
```



```
# Delete resurser
kubectl delete deployment todo-backend -n eks-mongo-todo
kubectl delete -f k8s/

# Scaling
kubectl scale deployment todo-backend --replicas=3 -n eks-mongo-todo

# Rolling restart
kubectl rollout restart deployment/todo-backend -n eks-mongo-todo
kubectl rollout status deployment/todo-backend -n eks-mongo-todo
kubectl rollout undo deployment/todo-backend -n eks-mongo-todo

# Context management
kubectl config get-contexts
kubectl config use-context arn:aws:eks:eu-west-1:xxx:cluster/todo-eks
```

3.4 Monitoring och Debugging

Kluster-hälsa:

```
# Node status
kubectl get nodes
kubectl top nodes # Kräver metrics-server

# Pod status
kubectl get pods --all-namespaces
kubectl top pods -n eks-mongo-todo

# Events
kubectl get events -n eks-mongo-todo --sort-by='.lastTimestamp'

# Cluster info
kubectl cluster-info
kubectl version
```

Debugging:

```
# Varför startar inte pod?
kubectl describe pod <pod-name> -n eks-mongo-todo

# Vanliga problem:
# - ImagePullBackOff: Image finns inte eller auth problem
# - CrashLoopBackOff: Container kraschar vid start
# - Pending: Ingen node har resurser
# - Error: Container exit code != 0

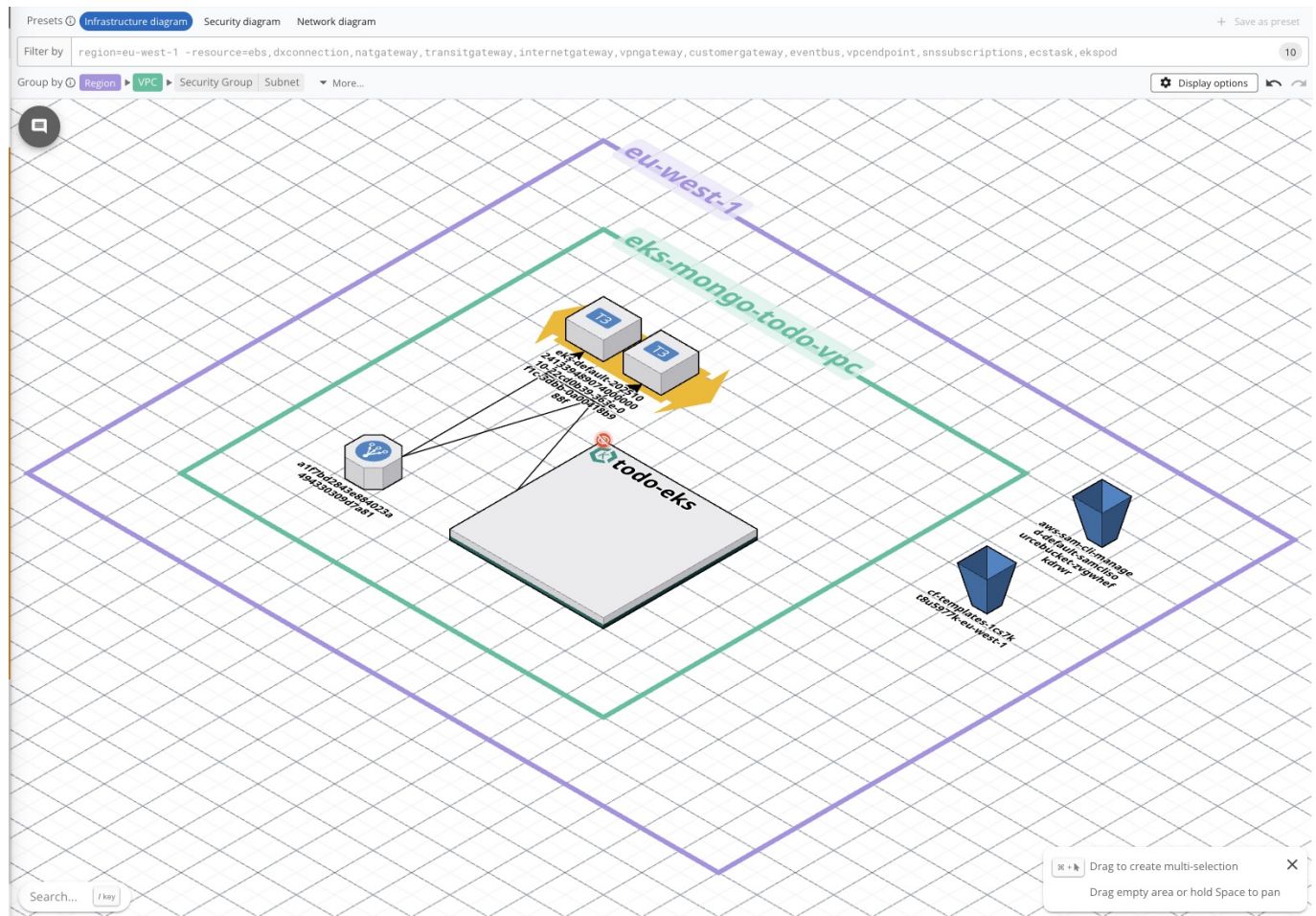
# Testa connectivity
```

```
kubectl run -it --rm debug --image=busybox --restart=Never -- sh
# Inne i pod:
wget -O- http://todo-backend/api/health
nslookup mongodb
```

4. Todo Application - Design och Deployment

4.1 Applikationsdesign

Arkitektur Overview



Hög nivå visar trafik från internet via NLB och Ingress till EKS-klustrets frontend, backend och MongoDB.

Komponenter:

1. Frontend (React + TypeScript + SASS)

- Single Page Application
- Axios för API-anrop
- Komponentbaserad arkitektur
- SASS variables för styling
- Responsive design

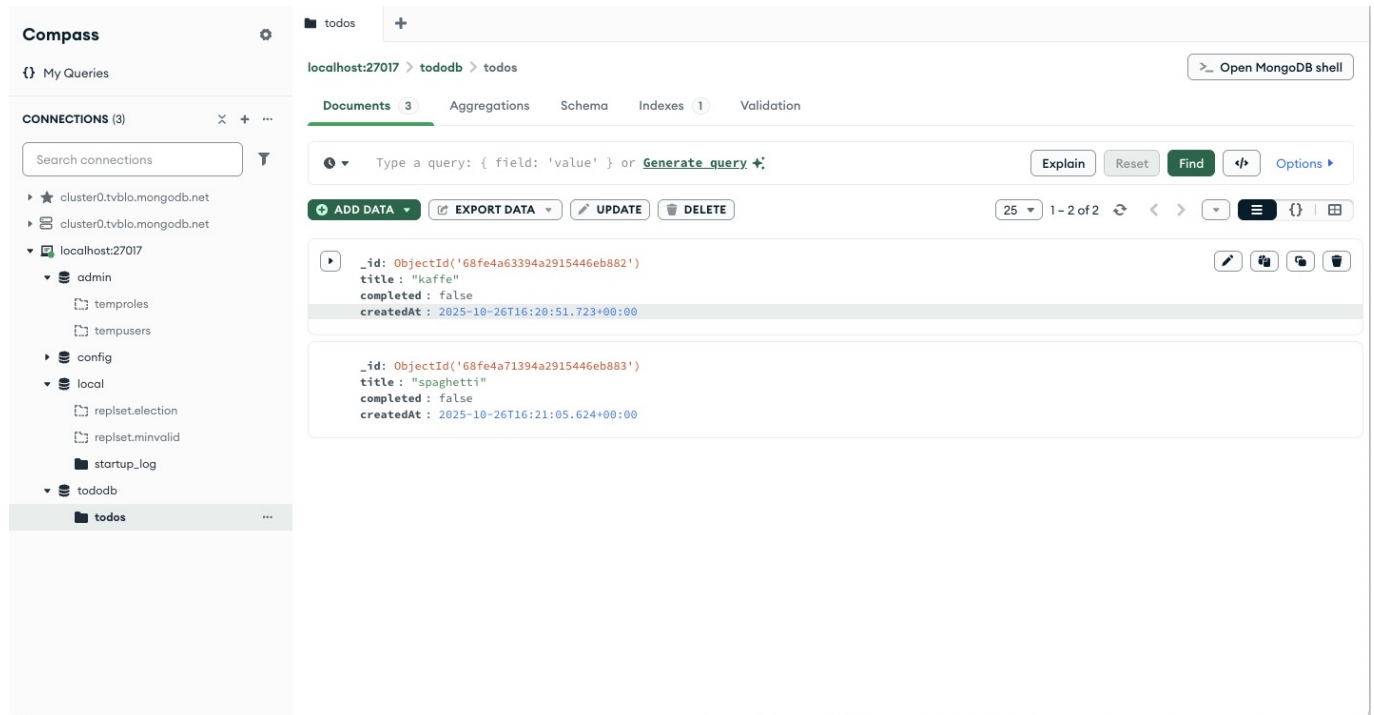
2. Backend (.NET 9 Web API)

- RESTful API

- MongoDB.Driver för databas-access
- CORS-konfiguration
- Minimal API pattern

3. Database (MongoDB 7)

- NoSQL document database
- Persistent storage via EBS
- StatefulSet för stable identity



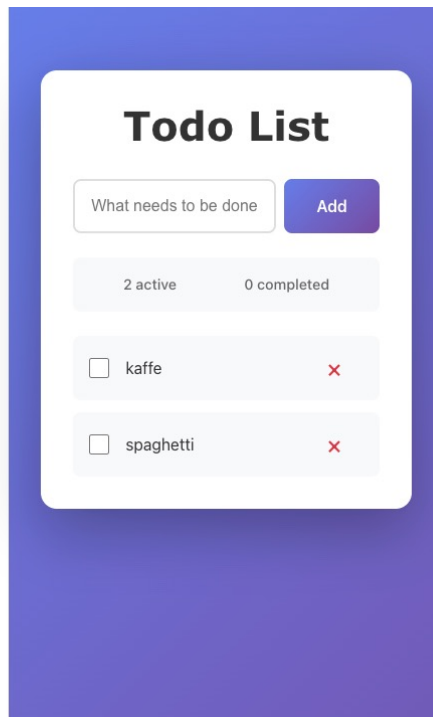
MongoDB Atlas/Compass visar databasens *todos*-samlingar och dokument som lagrar uppgifterna.

Dataflöde:

```

User → Browser → NLB → Ingress Controller → Frontend Service → Frontend Pod
                                     ↓
                                     Backend Service → Backend Pod →
MongoDB Service → MongoDB Pod
↓
EBS Volume (5Gi)

```



Publikt NLB-endpoint presenterar React-frontenden med två aktiva uppgifter.

4.2 Applikationskod

Frontend Struktur

```
app/todo-frontend/
├── src/
│   ├── components/
│   │   ├── TodoForm.tsx      # Input form för nya todos
│   │   ├── TodoItem.tsx     # Enskild todo rad
│   │   ├── TodoList.tsx     # Lista av todos
│   │   └── TodoStats.tsx    # Statistik (active/completed)
│   ├── services/
│   │   └── todoService.ts    # API calls med Axios
│   ├── styles/
│   │   └── _variables.scss   # SASS variables
│   ├── App.tsx              # Main component
│   └── main.tsx             # Entry point
├── Dockerfile
└── nginx.conf               # NGINX config för SPA routing
```

Exempel - TodoService:

```
import axios from "axios";

export type Todo = {
  id: string;
  title: string;
  completed: boolean;
```

```

    createdAt: string;
  };

  const API_BASE = import.meta.env.VITE_API_BASE_URL || "/api";

  export const todoService = {
    getAll: async (): Promise<Todo[]> => {
      const response = await axios.get<Todo[]>(`${API_BASE}/todos`);
      return response.data || [];
    },

    create: async (title: string): Promise<Todo> => {
      const response = await axios.post<Todo>(`${API_BASE}/todos`, { title
    });
      return response.data;
    },

    toggle: async (id: string, completed: boolean): Promise<void> => {
      await axios.put(`${API_BASE}/todos/${id}`, { completed: !completed });
    },

    delete: async (id: string): Promise<void> => {
      await axios.delete(`${API_BASE}/todos/${id}`);
    },
  };
};

```

Backend Struktur

```

app/todo-backend/
├── Program.cs           # Main API file
├── todo-backend.csproj  # Project dependencies
└── Dockerfile

```

Exempel - Backend API:

```

using MongoDB.Bson;
using MongoDB.Driver;

var builder = WebApplication.CreateBuilder(args);

// CORS
builder.Services.AddCors(options => {
    options.AddDefaultPolicy(policy => {
        policy.AllowAnyOrigin().AllowAnyMethod().AllowAnyHeader();
    });
});

// MongoDB
var mongoUri = builder.Configuration["MONGO_URI"]

```

```

    ?? "mongodb://root:changeme@mongodb:27017/?authSource=admin";
var mongoClient = new MongoClient(mongoUri);
var database = mongoClient.GetDatabase("tododb");
var todosCollection = database.GetCollection<TodoItem>("todos");

builder.Services.AddSingleton(todosCollection);

var app = builder.Build();
app.UseCors();

// Endpoints
app.MapGet("/api/todos", async (IMongoCollection<TodoItem> collection) =>
{
    var todos = await collection.Find(_ => true).ToListAsync();
    return Results.Ok(todos);
});

app.MapPost("/api/todos", async (TodoCreateDto dto,
IMongoCollection<TodoItem> collection) => {
    var todo = new TodoItem {
        Title = dto.Title,
        Completed = false,
        CreatedAt = DateTime.UtcNow
    };
    await collection.InsertOneAsync(todo);
    return Results.Created($"/api/todos/{todo.Id}", todo);
});

app.Run();

record TodoCreateDto(string Title);
class TodoItem {
    [BsonId]
    [BsonRepresentation(BsonType.ObjectId)]
    public string? Id { get; set; }
    public string Title { get; set; } = string.Empty;
    public bool Completed { get; set; }
    public DateTime CreatedAt { get; set; }
}

```

Administration lokalt och i molnet

4.3 Kubernetes Manifest Definitioner

Namespace

```

# k8s/namespace.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: eks-mongo-todo

```

```
labels:
  app.kubernetes.io/managed-by: terraform
  project: eks-mongo-todo
  owner: andreasvilhelmsson
  environment: dev
```

StorageClass

```
# k8s/storageclass-ebs-gp3.yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ebs-gp3
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
  encrypted: "true"
volumeBindingMode: WaitForFirstConsumer
```

Förklaring:

- Storage class fungerar som en "mall" för dynamisk lagring. beskriver hur och vilken typ av lagring som ska skapas automatiskt när en Pod begär en PersistentVolumeClaim.
- `provisioner: ebs.csi.aws.com` - Använder AWS EBS CSI Driver
- `type: gp3` - Senaste generation EBS (billigare än gp2)
- `encrypted: "true"` - Krypterar data at rest
- `WaitForFirstConsumer` - Skapar volume i samma AZ som pod

MongoDB StatefulSet och Service

```
# k8s/mongo/service.yaml
apiVersion: v1
kind: Service
metadata:
  name: mongodb
  namespace: eks-mongo-todo
spec:
  clusterIP: None # Headless service
  selector:
    app: mongodb
  ports:
    - port: 27017
```

```
# k8s/mongo/statefulset.yaml
apiVersion: apps/v1
```

```
kind: StatefulSet
metadata:
  name: mongodb
  namespace: eks-mongo-todo
spec:
  serviceName: mongodb
  replicas: 1
  selector:
    matchLabels:
      app: mongodb
  template:
    metadata:
      labels:
        app: mongodb
    spec:
      containers:
        - name: mongo
          image: mongo:7
          env:
            - name: MONGO_INITDB_ROOT_USERNAME
              value: root
            - name: MONGO_INITDB_ROOT_PASSWORD
              value: changeme
          ports:
            - containerPort: 27017
          volumeMounts:
            - name: data
              mountPath: /data/db
      volumeClaimTemplates:
        - metadata:
            name: data
          spec:
            accessModes: ["ReadWriteOnce"]
            storageClassName: ebs-gp3
            resources:
              requests:
                storage: 5Gi
```

Förklaring:

- **clusterIP: None** - Headless service ger stable DNS: `mongodb-0.mongodb.eks-mongo-todo.svc.cluster.local`
- **volumeClaimTemplates** - Skapar automatiskt PVC för varje replica
- **ReadWriteOnce** - Volume kan bara mountas av en node åt gången
- StatefulSet garanterar ordnad start: `mongodb-0` först, sedan `mongodb-1`, etc.

Backend Deployment och Service

```
# k8s/backend/svc.yaml
apiVersion: v1
```



```
kind: Service
metadata:
  name: todo-backend
  namespace: eks-mongo-todo
spec:
  selector:
    app: todo-backend
  ports:
    - port: 80
      targetPort: 8080
```

```
# k8s/backend/deploy.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: todo-backend
  namespace: eks-mongo-todo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: todo-backend
  template:
    metadata:
      labels:
        app: todo-backend
    spec:
      containers:
        - name: api
          image: "%%IMAGE%%" # Ersätts av Terraform
          ports:
            - containerPort: 8080
          env:
            - name: ASPNETCORE_URLS
              value: "http://+:8080"
            - name: MONGO_URI
              value: "mongodb://root:changeme@mongodb:27017/?authSource=admin"
```

Förklaring:

- Service exponerar port 80 externt, routar till pod port 8080
- %%IMAGE%% ersätts av Terraform med faktisk image URL
- MONGO_URI använder service name `mongodb` för DNS resolution
- Deployment skapar ReplicaSet som hanterar pods

Frontend Deployment och Service

```
# k8s/frontend/svc.yaml
apiVersion: v1
kind: Service
metadata:
  name: todo-frontend
  namespace: eks-mongo-todo
spec:
  selector:
    app: todo-frontend
  ports:
    - port: 80
      targetPort: 80
```

```
# k8s/frontend/deploy.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: todo-frontend
  namespace: eks-mongo-todo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: todo-frontend
  template:
    metadata:
      labels:
        app: todo-frontend
    spec:
      containers:
        - name: web
          image: "%IMAGE%"
          ports:
            - containerPort: 80
```

Förklaring:

- NGINX serverar den byggda React-applikationen som statiska filer via port 80
- Inga environment-variabler behövs för API URL, eftersom frontend kommunicerar med backend via /api som hanteras av Ingress.

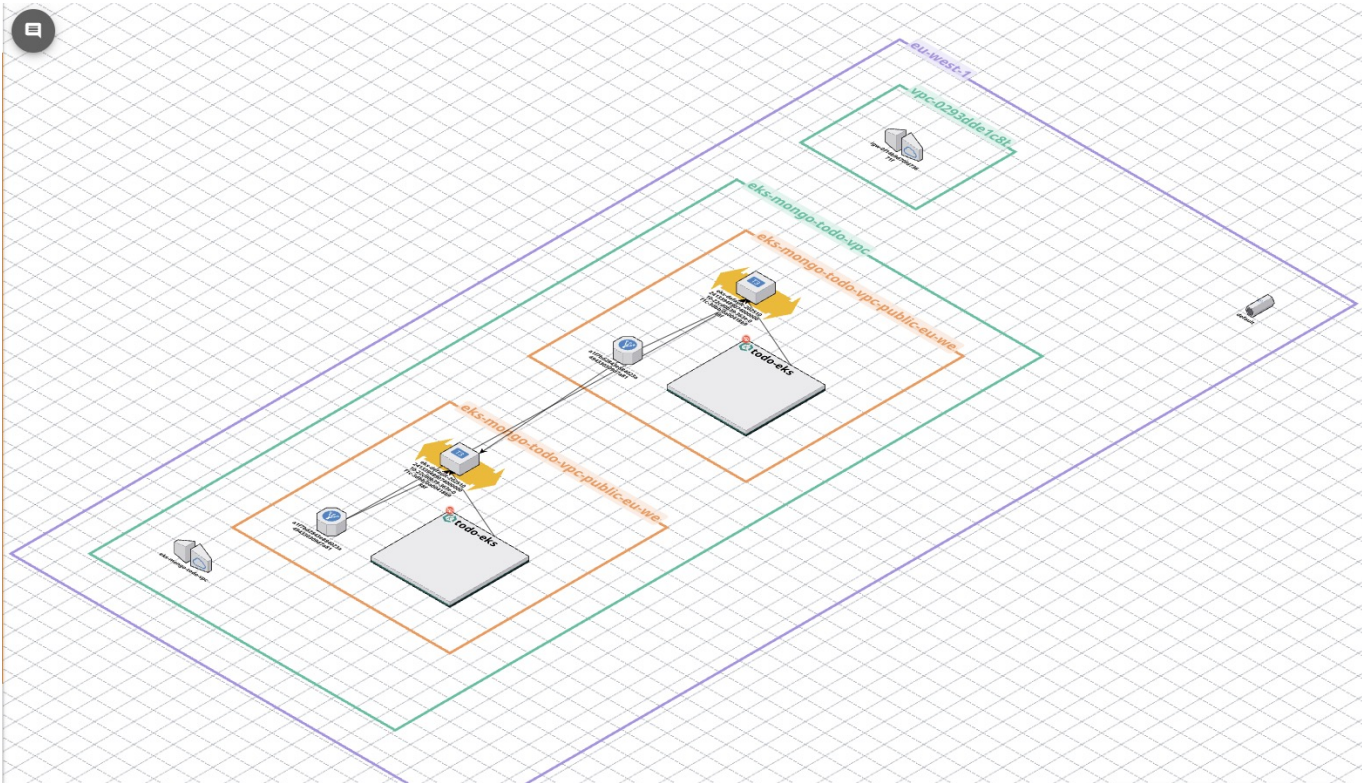
Ingress

```
# k8s/ingress/todo-ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
```

```
name: todo-ingress
namespace: eks-mongo-todo
annotations:
  nginx.ingress.kubernetes.io/proxy-connect-timeout: "600"
  nginx.ingress.kubernetes.io/proxy-send-timeout: "600"
  nginx.ingress.kubernetes.io/proxy-read-timeout: "600"
spec:
  ingressClassName: nginx
  rules:
    - http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: todo-backend
                port:
                  number: 80
          - path: /
            pathType: Prefix
            backend:
              service:
                name: todo-frontend
                port:
                  number: 80
```

Förklaring:

- `ingressClassName: nginx` - Använder NGINX Ingress Controller
- Path-based routing: `/api/*` → backend, `/*` → frontend
- `Prefix` pathType matchar alla sub-paths
- Timeout annotations för långsamma API calls
- Ingen rewrite - paths skickas som de är till backend



Visar hur Internettrafik går via ALB/NLB till Ingress, vidare till Services och respektive pods.

EC2 > Load balancers

EC2

Dashboards

EC2 Global View ^{L2}

Events

Instances

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Capacity Manager ^{New}

Images

AMIs

AMI Catalog

Elastic Block Store

Volumes

Snapshots

Lifecycle Manager

Network & Security

Security Groups

Elastic IPs

Introducing URL rewrite for Application Load Balancer

Modify host headers and URL paths of incoming requests before they reach your targets. To get started, add a rule to your listener and configure a transform.

×

Load balancers (1)

⌕

Filter load balancers

⌕

Elastic Load Balancing scales your load balancer capacity automatically in response to changes in incoming traffic.

⌕

Actions

Create load balancer

⌵

☐	Name	State	Type	Scheme	IP address type	VPC ID	Availability Zones
☐	a1f7bd2843e884023a494330309d7a81	-	classic	-	-	vpc-0cb677ba4aa5252a0	2 Availability Zones

0 load balancers selected

CloudShell

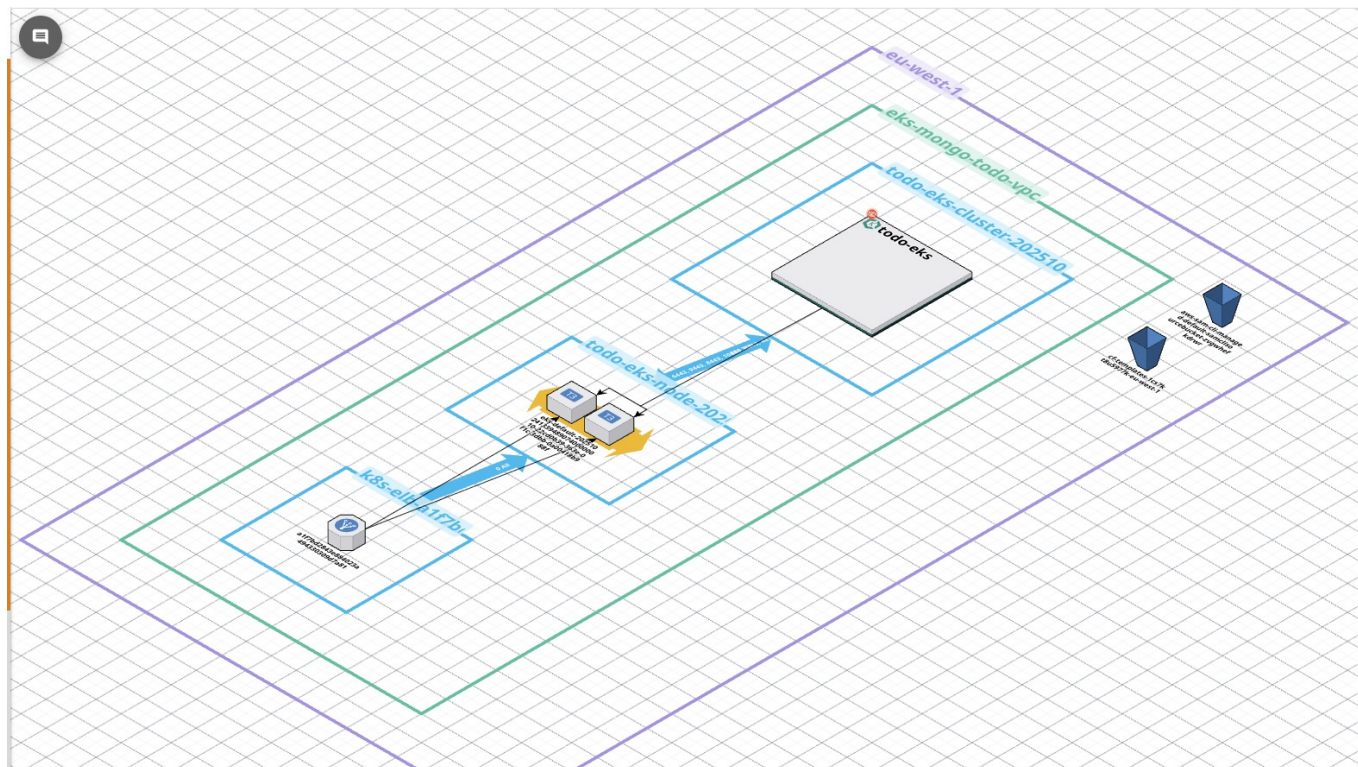
Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. [Privacy](#) [Terms](#) [Cookie preferences](#)

AWS Load Balancer Console visar NLB som skapats av Ingress Controller.

4.4 Säkerhetsdesign

28 / 52



Systemets säkerhet bygger på flera lager av åtkomstkontroller: • IAM + IRSA används för att ge specifika Kubernetes-poddar åtkomst till AWS-resurser (t.ex. EBS, S3) utan att ge hela noder fulla rättigheter. Detta följer principen "least privilege". • RBAC (Role-Based Access Control) styr vilka användare och tjänster som får göra förändringar i Kubernetes-klustret, t.ex. skapa pods, secrets eller uppdatera deployments. • Security Groups fungerar som virtuella brandväggar runt noder och lastbalanserare och kontrollerar vilken trafik som är tillåten in och ut ur AWS-infrastrukturen. • Tillsammans begränsar dessa mekanismer både nätverkstrafik och resursåtkomst, vilket minskar risken för obehörig åtkomst eller lateral movement inom systemet.

4.4.1 Nätverkssäkerhet

VPC och Subnets:

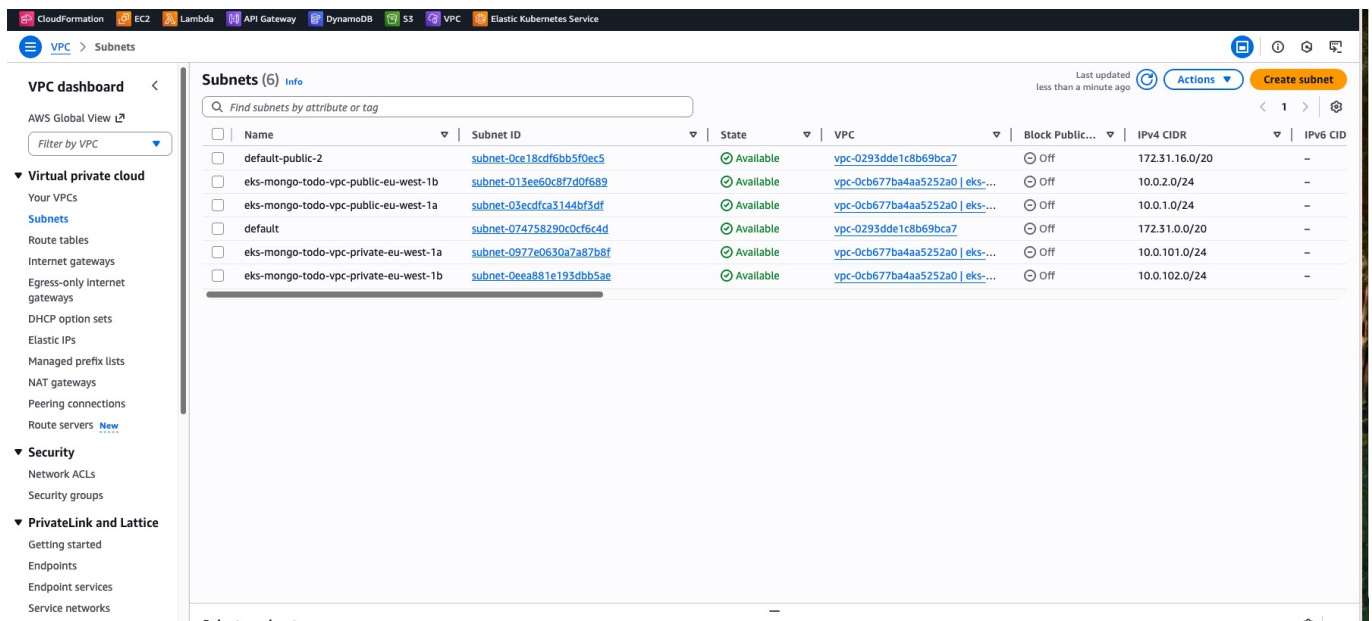
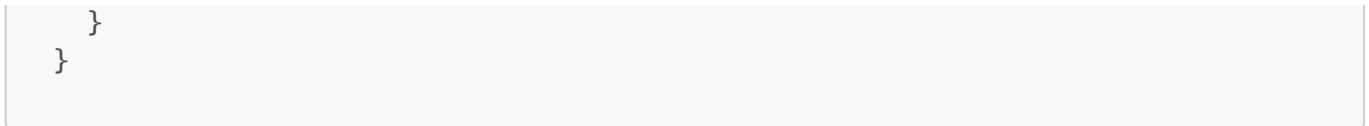
```
# infra/terraform/vpc.tf
module "vpc" {
  source = "terraform-aws-modules/vpc/aws"

  name = "eks-mongo-todo-vpc"
  cidr = "10.0.0.0/16"

  azs          = ["eu-west-1a", "eu-west-1b"]
  public_subnets = ["10.0.1.0/24", "10.0.2.0/24"]
  private_subnets = ["10.0.101.0/24", "10.0.102.0/24"]

  enable_nat_gateway = false # Cost optimization

  # EKS subnet tags
  public_subnet_tags = {
    "kubernetes.io/role/elb" = "1"
    "kubernetes.io/cluster/todo-eks" = "shared"
  }
}
```

AWS VPC Console visar public subnets med nödvändiga Kubernetes-taggar.

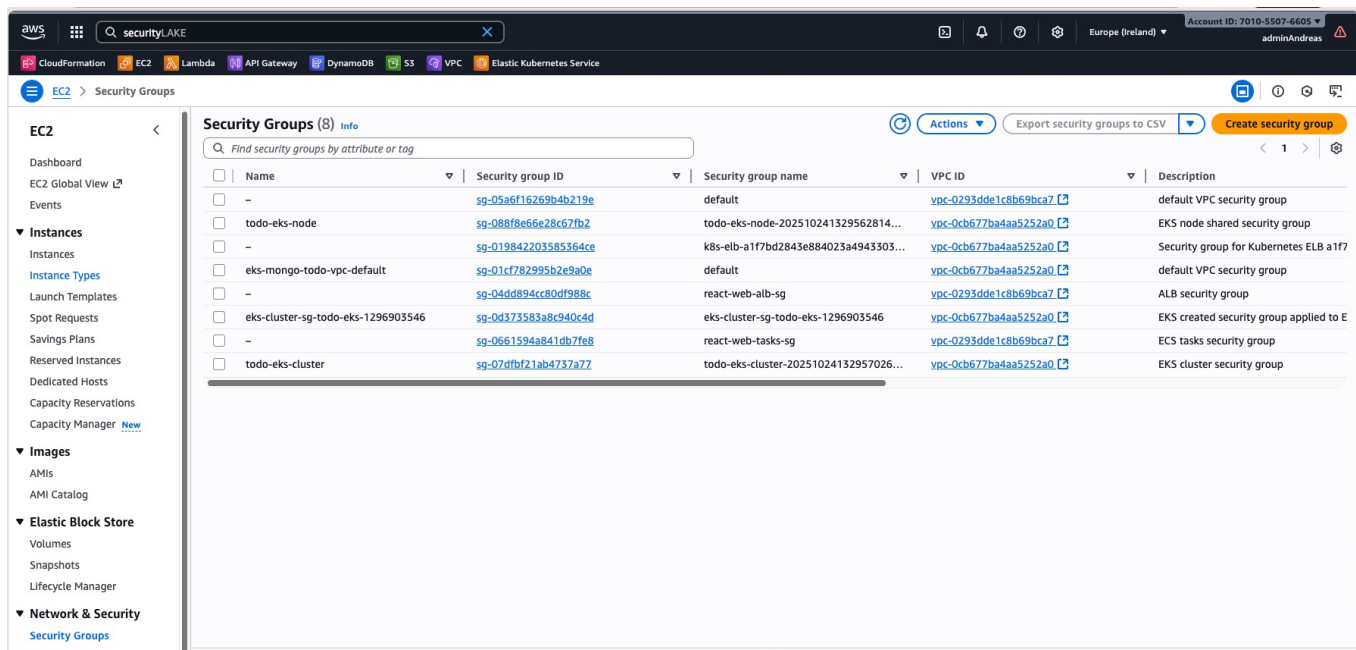
Security Groups:

1. Cluster Security Group (skapad av EKS)

- Tillåter kommunikation mellan control plane och worker nodes
- Port 443 för API Server

2. Node Security Group

```
# Tillåt all trafik mellan nodes (för pod-to-pod communication)
aws ec2 authorize-security-group-ingress \
  --group-id sg-088f8e66e28c67fb2 \
  --protocol all \
  --source-group sg-088f8e66e28c67fb2
```



VPC Console visar node security group med inbound-regler för pod-till-pod-trafik.

Network Policies (ej implementerat): NetworkPolicies har inte implementerats i detta projekt, men i en produktionsmiljö bör de användas för att begränsa nätverkstrafiken mellan pods. Med NetworkPolicies kan man definiera vilka pods eller namespaces som får kommunicera med varandra och vilken trafik (ingress/egress) som är tillåten. Detta minskar risken för lateral rörelse vid ett intrång och följer principen zero trust networking.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: backend-policy
  namespace: eks-mongo-todo
spec:
  podSelector:
    matchLabels:
      app: todo-backend
  policyTypes:
    - Ingress
    - Egress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: ingress-nginx
      ports:
        - protocol: TCP
          port: 8080
  egress:
    - to:
        - podSelector:
            matchLabels:
              app: mongodb
      ports:
```

```
- protocol: TCP  
  port: 27017
```

4.4.2 IAM och RBAC

EBS CSI Driver IAM Role (IRSA):

```
# infra/terraform/addons-ingress.tf  
data "aws_iam_policy_document" "ebs_csi_assume_role" {  
  statement {  
    actions = ["sts:AssumeRoleWithWebIdentity"]  
    principals {  
      type       = "Federated"  
      identifiers = [module.eks.oidc_provider_arn]  
    }  
    condition {  
      test     = "StringEquals"  
      variable = "${replace(module.eks.cluster_oidc_issuer_url,  
"https://", "")}:sub"  
      values   = ["system:serviceaccount:kube-system:ebs-csi-controller-  
sa"]  
    }  
  }  
}  
  
resource "aws_iam_role" "ebs_csi" {  
  name               = "todo-eks-ebs-csi-driver"  
  assume_role_policy =  
data.aws_iam_policy_document.ebs_csi_assume_role.json  
}  
  
resource "aws_iam_role_policy_attachment" "ebs_csi" {  
  role       = aws_iam_role.ebs_csi.name  
  policy_arn = "arn:aws:iam::aws:policy/service-  
role/AmazonEBSCSIDriverPolicy"  
}
```

Förklaring:

- IRSA (IAM Roles for Service Accounts) - Kubernetes ServiceAccount kan assume IAM role
- EBS CSI Driver behöver permissions för CreateVolume, AttachVolume, etc.
- Ingen access keys i pods - säkrare än node IAM role

Role name	Trusted entities	Last activity
AWSServiceRoleForAmazonEKS	AWS Service: eks (Service-Linked Role)	8 minutes ago
AWSServiceRoleForAmazonEKSNodegroup	AWS Service: eks-nodegroup (Service-Linked Role)	11 minutes ago
AWSServiceRoleForAmazonElasticFileSystem	AWS Service: elasticfilesystem (Service-Linked Role)	48 days ago
AWSServiceRoleForAPIGateway	AWS Service: ops.apigateway (Service-Linked Role)	-
AWSServiceRoleForAutoScaling	AWS Service: autoscaling (Service-Linked Role)	8 minutes ago
AWSServiceRoleForEC2Spot	AWS Service: spot (Service-Linked Role)	-
AWSServiceRoleForECS	AWS Service: ecs (Service-Linked Role)	23 days ago
AWSServiceRoleForElasticLoadBalancing	AWS Service: elasticloadbalancing (Service-Linked Role)	Yesterday
AWSServiceRoleForRDS	AWS Service: rds (Service-Linked Role)	35 minutes ago
AWSServiceRoleForResourceExplorer	AWS Service: resource-explorer-2 (Service-Linked Role)	13 minutes ago
AWSServiceRoleForSupport	AWS Service: support (Service-Linked Role)	4 days ago
AWSServiceRoleForTrustedAdvisor	AWS Service: trustedadvisor (Service-Linked Role)	-
CloudcraftReadOnlyRole	Account: 968898580625	34 days ago
default-eks-node-group-20251024132944364200000001	AWS Service: ec2	10 minutes ago
MyAmazonEKSAutoClusterRole	AWS Service: eks	-
rds-monitoring-role	AWS Service: monitoring.rds	-
react-web-exec-role	AWS Service: ecs-tasks	23 days ago
react-web-task-role	AWS Service: ecs-tasks	-
serverless-contact-form-ApiFnRole-GW6zpU7mOUHT	AWS Service: lambda	19 days ago
todo-eks-cluster-20251024132944364200000002	AWS Service: eks	4 minutes ago

AWS IAM Console visar rollen som används av EBS CSI Driver via IRSA.

Kubernetes RBAC (ej implementerat): För production skulle vi skapa ServiceAccounts med begränsade permissions:

```

apiVersion: v1
kind: ServiceAccount
metadata:
  name: backend-sa
  namespace: eks-mongo-todo
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: backend-role
  namespace: eks-mongo-todo
rules:
- apiGroups: [""]
  resources: ["configmaps", "secrets"]
  verbs: ["get", "list"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: backend-rolebinding
  namespace: eks-mongo-todo
subjects:
- kind: ServiceAccount
  name: backend-sa
roleRef:
  kind: Role

```

```
name: backend-role
apiGroup: rbac.authorization.k8s.io
```

4.4.3 Secrets Management

Nuvarande implementation (ej production-ready):

```
env:
  - name: MONGO_URI
    value: "mongodb://root:changeme@mongodb:27017/?authSource=admin"
```

Bättre approach - Kubernetes Secrets:

```
apiVersion: v1
kind: Secret
metadata:
  name: mongo-credentials
  namespace: eks-mongo-todo
type: Opaque
stringData:
  username: root
  password: changeme
  uri: mongodb://root:changeme@mongodb:27017/?authSource=admin
---
# I Deployment:
env:
  - name: MONGO_URI
    valueFrom:
      secretKeyRef:
        name: mongo-credentials
        key: uri
```

Best practice - AWS Secrets Manager:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: backend-sa
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::xxx:role/backend-secrets-role
---
# I Deployment:
env:
  - name: MONGO_URI
    valueFrom:
      secretKeyRef:
```

```
name: mongo-credentials # Synced från AWS Secrets Manager
key: uri
```

Med External Secrets Operator eller AWS Secrets CSI Driver.

4.4.4 Container Security

Dockerfile Best Practices:

Backend Dockerfile:

```
FROM mcr.microsoft.com/dotnet/sdk:9.0 AS build
WORKDIR /src
COPY *.csproj .
RUN dotnet restore
COPY . .
RUN dotnet publish -c Release -o /app

FROM mcr.microsoft.com/dotnet/aspnet:9.0
WORKDIR /app
COPY --from=build /app .

# Security: Run as non-root
USER app
EXPOSE 8080
ENTRYPOINT ["dotnet", "todo-backend.dll"]
```

Frontend Dockerfile:

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
ARG VITE_API_BASE_URL=/api
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf

# Security: Run as non-root
USER nginx
EXPOSE 80
```

Security features:

- Multi-stage builds - mindre image size
- Alpine base images - färre vulnerabilities
- Non-root user - begränsar attack surface
- No secrets in images - environment variables istället

Image Scanning (ej implementerat): För production: Trivy eller AWS ECR image scanning

```
trivy image ghcr.io/andreasvilhelmsson/todo-backend-v2:latest
```

4.4.5 Data Encryption

At Rest:

- EBS volumes encrypted via StorageClass: `encrypted: "true"`
- Managed by AWS KMS

In Transit:

- HTTPS via Ingress (skulle kräva TLS certificate)
- MongoDB connection utan TLS (skulle behöva konfigureras)

Production improvements:

```
# Ingress med TLS
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: todo-ingress
  annotations:
    cert-manager.io/cluster-issuer: "letsencrypt-prod"
spec:
  tls:
    - hosts:
      - todo.example.com
      secretName: todo-tls
  rules:
    - host: todo.example.com
      http:
        paths: [...]
```

Egen applikation på EKS

4.6 Deployment Process

4.6.1 CI/CD Pipeline med GitHub Actions

Workflow för Backend:

```
# .github/workflows/backend-image.yml
name: build-backend

on:
  push:
    paths:
      - app/todo-backend/**
      - .github/workflows/backend-image.yml

permissions:
  contents: read
  packages: write

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-dotnet@v4
        with: { dotnet-version: 9.0.x }

      - name: dotnet restore
        working-directory: app/todo-backend
        run: dotnet restore

      - name: dotnet build
        working-directory: app/todo-backend
        run: dotnet build --configuration Release --no-restore

      - name: Extract metadata
        id: meta
        uses: docker/metadata-action@v5
        with:
          images: ghcr.io/andreasvilhelmsson/todo-backend-v2
          tags: |
            type=raw,value=0.1.${{ github.run_number }}
            type=raw,value=latest,enable=${{ github.ref ==
'refs/heads/main' }}

      - uses: docker/setup-buildx-action@v3

      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${ github.actor }
          password: ${ secrets.GITHUB_TOKEN }

      - uses: docker/build-push-action@v6
        with:
          context: ./app/todo-backend
          platforms: linux/amd64,linux/arm64
```

```
push: true
tags: ${ steps.meta.outputs.tags }
cache-from: type=gha,scope=todo-backend
cache-to: type=gha,mode=max,scope=todo-backend
```

Workflow för Frontend:

```
# .github/workflows/frontend-image.yml
name: build-frontend

on:
  push:
    paths:
      - app/todo-frontend/**
      - .github/workflows/frontend-image.yml

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm
          cache-dependency-path: app/todo-frontend/package-lock.json

      - name: npm ci
        working-directory: app/todo-frontend
        run: npm ci

      - name: npm build
        working-directory: app/todo-frontend
        env:
          VITE_API_BASE_URL: /api
        run: npm run build

      - uses: docker/build-push-action@v6
        with:
          context: ./app/todo-frontend
          platforms: linux/amd64,linux/arm64
          push: true
          tags: ghcr.io/andreasvilhelmsson/todo-frontend-v2:latest
          build-args: VITE_API_BASE_URL=/api
```

Fördelar med denna pipeline:

- Automatisk build vid code push
- Multi-arch images (amd64 + arm64)

- Layer caching för snabba builds
 - Semantic versioning: `0.1.<run_number>`
 - Preflight testing (dotnet build, npm build) innan Docker build
-

4.6.2 Infrastructure as Code med Terraform

Terraform Struktur:

```
infra/terraform/  
├── versions.tf          # Provider versions och backend  
├── providers.tf         # AWS provider config  
├── providers-k8s.tf     # Kubernetes/Helm providers  
├── variables.tf         # Input variables  
├── outputs.tf           # Output values  
├── vpc.tf              # VPC module  
├── eks.tf              # EKS cluster module  
├── addons-ingress.tf   # EBS CSI + Ingress  
└── apply-k8s.tf        # Kubernetes manifests
```

Deployment Steps:

Steg 1: Skapa infrastruktur

```
cd infra/terraform  
  
# Initiera Terraform  
terraform init  
  
# Validera konfiguration  
terraform validate  
  
# Planera ändringar  
terraform plan  
  
# Applicera  
terraform apply
```

Output:

```
Apply complete! Resources: 51 added, 0 changed, 0 destroyed.  
  
Outputs:  
cluster_endpoint = "https://xxx.gr7.eu-west-1.eks.amazonaws.com"  
cluster_name = "todo-eks"  
vpc_id = "vpc-0cb677ba4aa5252a0"  
public_subnets = ["subnet-03ecdfca3144bf3df", "subnet-013ee60c8f7d0f689"]
```

Steg 2: Konfigurera kubectl

```
aws eks update-kubeconfig --region eu-west-1 --name todo-eks

# Verifiera
kubectl get nodes
kubectl get pods --all-namespaces
```

Steg 3: Installera Ingress Controller

```
# Lägg till Helm repo
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo update

# Installera
helm install ingress-nginx ingress-nginx/ingress-nginx \
  -n ingress-nginx \
  --create-namespace \
  --wait=false

# Vänta på Load Balancer
kubectl get svc -n ingress-nginx ingress-nginx-controller -w
```

Steg 4: Deploya applikation

```
# Applicera alla manifests
kubectl apply -f k8s/

# Eller via Terraform (redan gjort i steg 1)
# Terraform skapar kubernetes_manifest resurser automatiskt
```

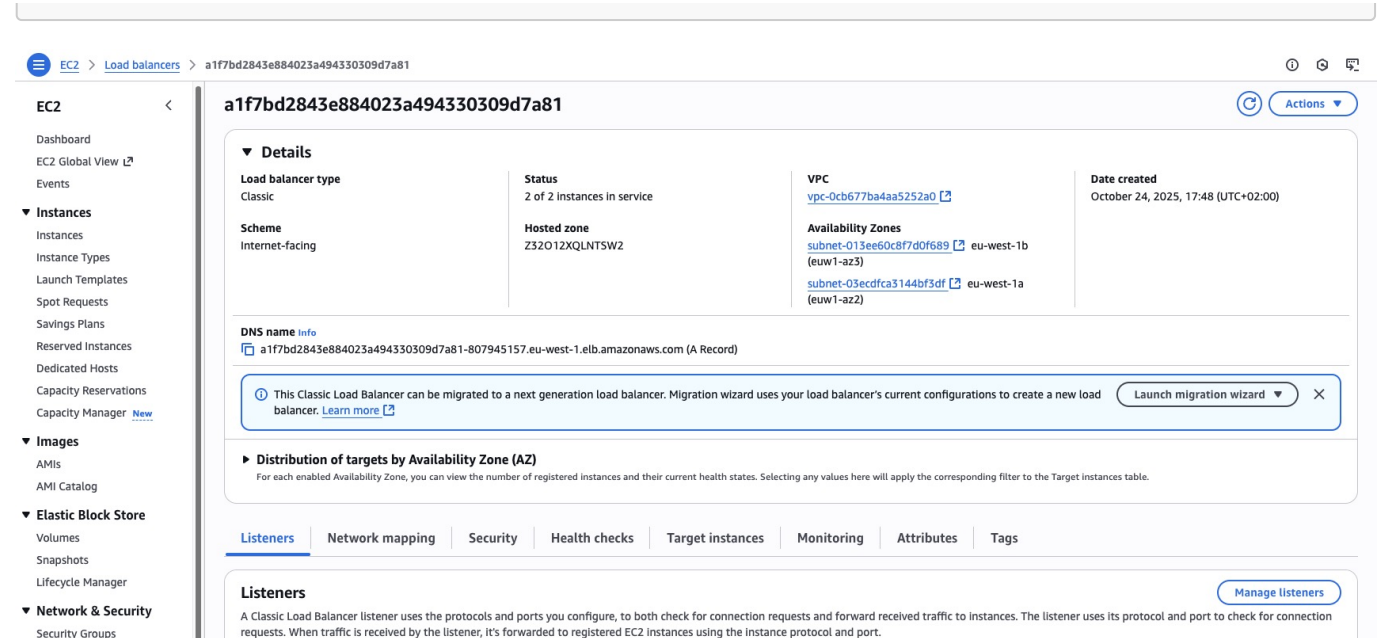
Steg 5: Verifiera deployment

```
# Kolla pods
kubectl get pods -n eks-mongo-todo

# Kolla services
kubectl get svc -n eks-mongo-todo

# Kolla ingress
kubectl get ingress -n eks-mongo-todo

# Hämta Load Balancer URL
kubectl get svc -n ingress-nginx ingress-nginx-controller \
  -o jsonpath='{.status.loadBalancer.ingress[0].hostname}'
```

AWS Load Balancer Console visar listeners och target groups för ingresskontrollern.

4.6.3 Uppdatera Applikation

Scenario: Ny feature i frontend

Steg 1: Utveckla och testa lokalt

```
cd app/todo-frontend
npm run dev

# Testa ändringar
# Commit när klar
```

Steg 2: Push till GitHub

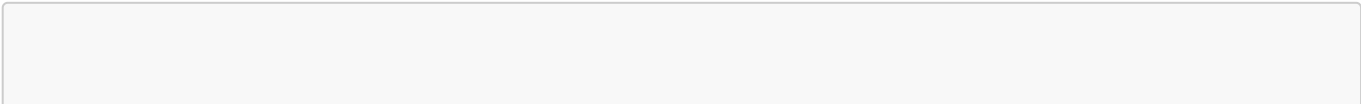
```
git add app/todo-frontend/
git commit -m "feat: Add new todo feature"
git push origin main
```

Steg 3: GitHub Actions bygger automatiskt

- Workflow triggas av push
- Bygger ny image: ghcr.io/andreasvilhelmsson/todo-frontend-v2:0.1.42
- Pushar till GitHub Container Registry

Steg 4: Uppdatera Kubernetes

Alternativ A: Rolling restart (använder :latest tag)



```
kubectl rollout restart deployment/todo-frontend -n eks-mongo-todo

# Följ status
kubectl rollout status deployment/todo-frontend -n eks-mongo-todo

# Verifiera
kubectl get pods -n eks-mongo-todo
```

Alternativ B: Uppdatera image tag (bättre för production)

```
kubectl set image deployment/todo-frontend \
  web=ghcr.io/andreasvilhelmsson/todo-frontend-v2:0.1.42 \
  -n eks-mongo-todo

# Eller via Terraform
terraform apply -var="frontend_image=ghcr.io/andreasvilhelmsson/todo-
frontend-v2:0.1.42"
```

Rollback vid problem:

```
# Visa rollout history
kubectl rollout history deployment/todo-frontend -n eks-mongo-todo

# Rollback till föregående version
kubectl rollout undo deployment/todo-frontend -n eks-mongo-todo

# Rollback till specifik revision
kubectl rollout undo deployment/todo-frontend --to-revision=2 -n eks-
mongo-todo
```

4.6.4 Monitoring och Logging

Pod Logs:

```
# Visa logs
kubectl logs deployment/todo-backend -n eks-mongo-todo --tail=100

# Follow logs
kubectl logs -f deployment/todo-backend -n eks-mongo-todo

# Logs från alla pods med label
kubectl logs -l app=todo-backend -n eks-mongo-todo --all-containers=true
```

Events:

```
# Visa events i namespace
kubectl get events -n eks-mongo-todo --sort-by='.lastTimestamp'

# Watch events
kubectl get events -n eks-mongo-todo --watch
```

Resource Usage:

```
# Node resources
kubectl top nodes

# Pod resources
kubectl top pods -n eks-mongo-todo
```

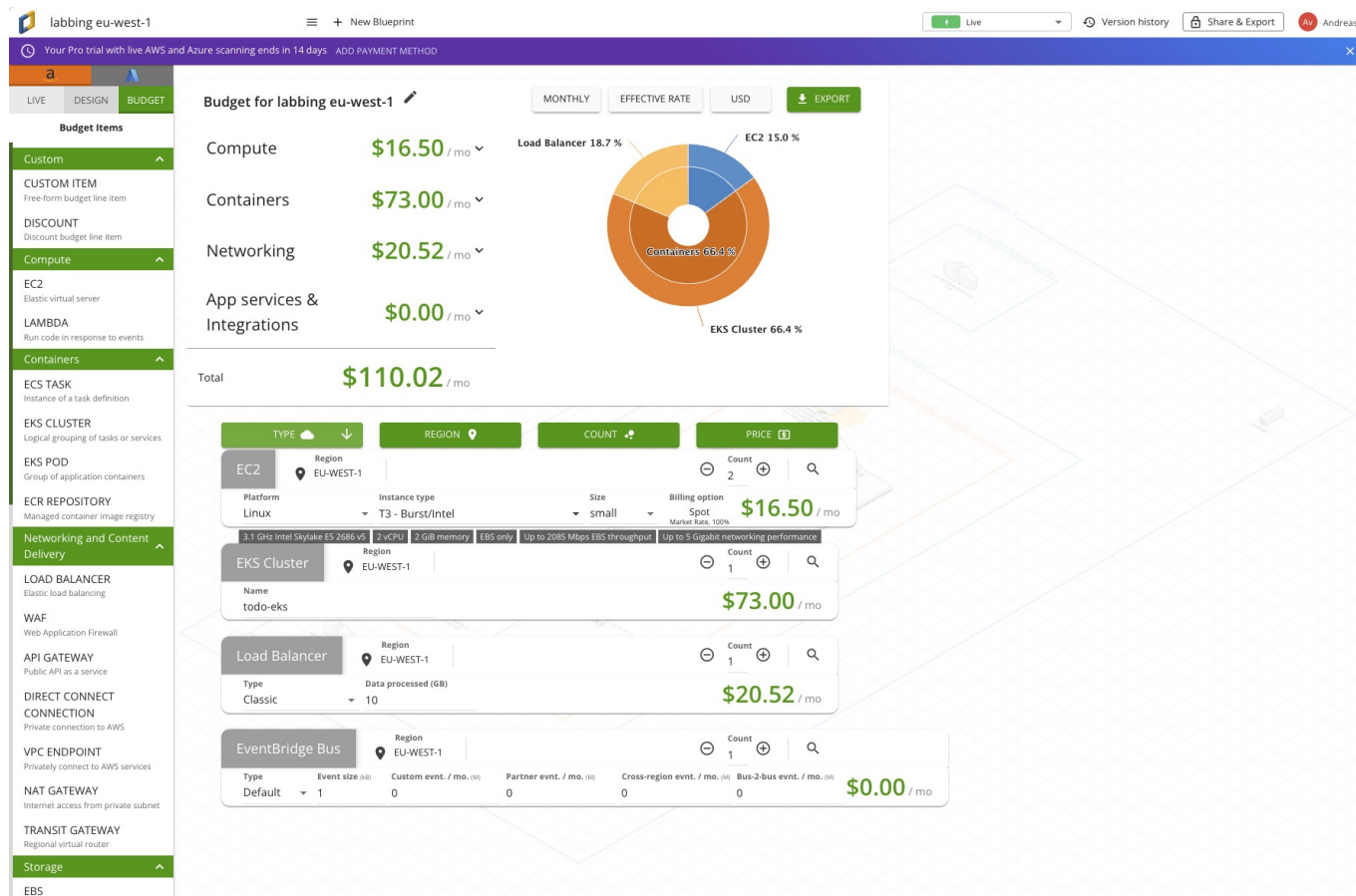
Production Monitoring (ej implementerat): För production skulle vi installera:

- Prometheus + Grafana för metrics
- ELK Stack eller CloudWatch för logs
- Jaeger för distributed tracing

4.7 Kostnadsanalys

Månadskostnad:

Resurs	Specifikation	Kostnad/månad
EKS Control Plane	Managed Kubernetes	\$73
EC2 Instances	2x t3.small spot	~\$10
EBS Volumes	5GB gp3	\$0.40
Network Load Balancer	1x NLB	~\$16
Data Transfer	Minimal (dev)	~\$1
Total		~\$100



Cost Explorer visar hur kostnaden fördelas mellan EKS, EC2, NLB och lagring.

Kostnadsoptimering:

- Spot instances istället för On-Demand (70% billigare)
- Ingen NAT Gateway (sparar \$32/månad)
- Single NLB för alla services via Ingress
- gp3 istället för gp2 (20% billigare)
- Minimal node count (2 för HA)

Skalning för production:

- 3+ nodes för HA: +\$15/månad
- NAT Gateway för private subnets: +\$32/månad
- Större instances (t3.medium): +\$20/månad
- Backup och monitoring: +\$10/månad
- **Production total: ~\$180/månad**

5. Utmaningar och Lösningar

5.1 Node Capacity Problem

Problem: Pods fastnade i Pending state med error "Too many pods".

Orsak: Single t3.small node kunde inte rymma alla system pods + application pods under rolling update.

Lösning: Ökade från 1 till 2 nodes. Krävde multi-step AWS CLI commands pga EKS constraints.

Lärdomar:

- Planera för rolling update overhead (2x pods under update)
 - t3.small kan rymma ~10-15 pods beroende på resource requests
 - Använd minst 2 nodes för production
-

5.2 Inter-Node Networking

Problem: 504 Gateway Timeout när Ingress Controller (node 1) försökte nå frontend pod (node 2).

Orsak: Node security group tillät inte all trafik mellan nodes. EKS skapar bara rules för specifika ports (kubelet, coredns).

Lösning:

```
aws ec2 authorize-security-group-ingress \
  --group-id sg-088f8e66e28c67fb2 \
  --protocol all \
  --source-group sg-088f8e66e28c67fb2
```

Lärdomar:

- Multi-node kluster kräver explicit security group konfiguration
 - Testa pod-to-pod communication mellan nodes
 - Dokumentera security group rules
 - Att göra en checklista beroende på de fel som man har råkat ut för på vägen
-

5.3 Ingress Path Rewriting

Problem: API calls returnerade 404 Not Found.

Orsak: Ingress-regeln ändrade URL:en genom att ta bort ("strippa") /api från början av sökvägen innan trafiken skickas vidare till backend-servicen.

```
nginx.ingress.kubernetes.io/rewrite-target: /$1
path: /api(/|$)(.*)
```

Detta skrev om /api/todos → /todos, men backend förväntade /api/todos.

Lösning: Tog bort rewrite och använde enkel prefix matching:

```
path: /api
pathType: Prefix
```

Lärdomar:

- Undvik komplexa regex rewrites om möjligt
 - Testa Ingress routing noggrant
 - Använd `kubectl logs` på Ingress Controller för debugging
-

5.4 EBS CSI Driver Timeout

Problem: EBS CSI add-on fastnade i DEGRADED state i 20+ minuter.

Orsak: Saknade IAM role med permissions för EBS operations.

Lösning: Skapade IAM role med IRSA (IAM Roles for Service Accounts):

```
resource "aws_iam_role" "ebs_csi" {
  name = "todo-eks-ebs-csi-driver"
  assume_role_policy =
data.aws_iam_policy_document.ebs_csi_assume_role.json
}

resource "aws_eks_addon" "ebs_csi" {
  service_account_role_arn = aws_iam_role.ebs_csi.arn
  ...
}
```

Lärdomar:

- EKS addons behöver ofta IAM roles
 - Använd IRSA istället för node IAM roles IRSA (IAM Roles for Service Accounts) är ett sätt för Kubernetes-poddar i EKS att få specifika AWS-rättigheter utan att ge hela noden åtkomst via en node IAM role. Detta ökar säkerheten, följer "least privilege"-principen och är AWS rekommenderade metod i moderna EKS-kluster.
 - Kolla add-on status: `kubectl get pods -n kube-system`
-

6. Slutsats

Projektet visar inte bara hur man kan deploya en modern fullstack-applikation till EKS med skalbarhet och automatisering – det visar även vilka fallgropar, beroenden och best practices som är avgörande i ett verkligt DevOps-flöde. Kombinationen av Terraform, Docker, Kubernetes och Helm gav ett robust ramverk, men krävde förståelse för IAM-säkerhet, lagring (EBS), nätverk (Ingress), versionskontroll och CI/CD-strategier. Att tolka och lösa alla felmeddelanden är en stor utmaning även med hjälp av LLMer

Viktiga lärdomar:

1. **Kubernetes är kraftfullt men komplext** Kubernetes erbjuder extrem flexibilitet och skalbarhet, men kräver god förståelse för objekt som Pods, Deployments, Services, Ingress och Volumes. Felkonfigurationer är vanliga i början och små misstag kan orsaka stora problem i hela klustret.
2. **Infrastructure as Code är essentiellt** Med Terraform kan all infrastruktur (VPC, EKS, IAM, nätverk) skapas, uppdateras och återställas på ett reproducerbart sätt. Versionering i Git möjliggör

spårbarhet, kodgranskning och enklare samarbete i team.

3. **Security är multi-layered** Säkerheten sker i flera lager: nätverkspolicies, IAM/IRSA för resurshantering, RBAC i Kubernetes och hantering av Secrets. Minsta möjliga behörighet (Least Privilege) och isolering mellan pods, namespaces och roller är avgörande.
4. **Monitoring är kritiskt** Utan korrekt loggning och metrics blir det nästan omöjligt att felsöka problem i distribuerade system. Verktyg som Prometheus, Grafana, CloudWatch och kubectl logs är nödvändiga för att övervaka hälsa, resurser och applikationsstatus.
5. **Cost optimization matters** Molntjänster kostar snabbt om man inte optimerar resurser. Genom att använda rätt instanstyper, autoscaling, spot-instances och ta bort oanvända resurser kan kostnader minimeras utan att påverka prestanda. Dessutom måste jag riva allt så fort jag är klar eftersom det är aldeles för dyrt för mig att ha uppe

Förbättringar för production:

- ☐ Implementera NetworkPolicies
- ☐ Använd AWS Secrets Manager för credentials
- ☐ Lägg till TLS/HTTPS via cert-manager
- ☐ Installera Prometheus + Grafana
- ☐ Konfigurera HorizontalPodAutoscaler
- ☐ Implementera proper RBAC
- ☐ Lägg till health checks och readiness probes
- ☐ Konfigurera resource requests/limits
- ☐ Sätt upp backup för MongoDB
- ☐ Implementera GitOps med ArgoCD

Resultat:

- Fungerande todo-applikation på AWS EKS
- Automatisk CI/CD med GitHub Actions
- Infrastructure as Code med Terraform
- Persistent storage med EBS
- Public access via Ingress och NLB
- Kostnadsoptimerad (~\$100/månad)

7. Referenser

Dokumentation:

- Kubernetes Official Docs: <https://kubernetes.io/docs/>
- AWS EKS User Guide: <https://docs.aws.amazon.com/eks/>
- Terraform AWS Provider: <https://registry.terraform.io/providers/hashicorp/aws/>
- NGINX Ingress Controller: <https://kubernetes.github.io/ingress-nginx/>

Terraform Modules:

- EKS Module: <https://registry.terraform.io/modules/terraform-aws-modules/eks/aws/>
- VPC Module: <https://registry.terraform.io/modules/terraform-aws-modules/vpc/aws/>

Tools:

- kubectl: <https://kubernetes.io/docs/tasks/tools/>
- Helm: <https://helm.sh/>
- Docker: <https://docs.docker.com/>

Git Repository: <https://github.com/andreasvilhelmsson/eks-mongo-todo>

Arkitekturdiagram (Mermaid)

```

flowchart LR
    subgraph AWS_VPC["AWS VPC (eu-west-1)"]
        subgraph Public_Subnets["Public subnets"]
            alb[ALB/NLB]
        end
        subgraph Private_Subnets["Private subnets"]
            eks[[EKS Cluster]]
            subgraph NamespaceEks["Namespace: eks-mongo-todo"]
                fe[Frontend Deployment]
                be[Backend Deployment]
                db[[MongoDB StatefulSet + PVC/EBS]]
            end
        end
    end

    User((User)) -->|HTTP| alb
    alb --> fe
    fe -->|/api| be
    be --> db

    eks --> fe
    eks --> be
    eks --> db

    gh[GitHub Actions] --> ghcr[[GitHub Container Registry]]
    ghcr --> eks

```

Lärdomar och fallgropar**5.5 VPC Configuration Typos**

Problem: Terraform apply misslyckades med "unknown variable" errors.

Orsak: Typos i vpc.tf - `vpc_deor` istället för `vpc_cidr`, `gats` istället för `tags`.

Lösning: Korrigerade variabelnamn och tog bort `enable_dns64` (ej supporterat i VPC module v5).

Lärdomar:

- Använd IDE med syntax highlighting
- Kör `terraform validate` innan apply

- Läs module documentation noggrant
-

5.6 MongoDB Multi-Document YAML

Problem: Terraform kunde inte parse MongoDB YAML med `---` separator.

Orsak: `yamldecode()` function stödjer inte multi-document YAML.

Lösning: Splittade i separata filer:

- `k8s/mongo/service.yaml`
- `k8s/mongo/statefulset.yaml`

Lärdomar:

- Terraform `yamldecode` kräver single-document YAML
 - Separata filer ger bättre struktur ändå
 - Använd `kubectl apply -f directory/` för multi-file deploys
-

5.7 SASS Import Syntax Deprecation

Problem: Frontend build misslyckades med "@import is deprecated" warnings.

Orsak: Modern SASS kräver `@use` istället för `@import`.

Lösning: Uppdaterade alla SASS-filer:

```
// Gammalt
@import "variables";

// Nytt
@use "variables" as *;
```

Lärdomar:

- Håll dig uppdaterad med framework deprecations
 - `@use` ger bättre namespace-hantering
 - Testa builds lokalt innan push
-

5.8 TypeScript verbatimModuleSyntax

Problem: TypeScript compilation errors: "'type' modifier cannot be used on a named export".

Orsak: `verbatimModuleSyntax: true` kräver explicit `import type` syntax.

Lösning:

```
// Gammalt
import { Todo } from "./types";

// Nytt
import type { Todo } from "./types";
```

Lärdomar:

- TypeScript 5.0+ har striktare type import rules
 - `verbatimModuleSyntax` förbättrar tree-shaking
 - Använd ESLint rule för att enforcea korrekt syntax
-

5.9 Helm Provider Version Conflicts

Problem: Terraform kunde inte initiera Helm provider - syntax errors.

Orsak: Blandade Helm provider v2 och v3 syntax.

Lösning: Använde tom Helm provider block som ärver från Kubernetes provider:

```
provider "helm" {
  # Inherits from kubernetes provider
}
```

Lärdomar:

- Helm provider v2.12+ kan ärva kubernetes config
 - Undvik duplicerad konfiguration
 - Testa provider init separat: `terraform init -upgrade`
-

5.10 GitHub Actions Multi-Arch Builds

Problem: Images byggda på GitHub Actions (AMD64) fungerade inte lokalt (ARM64 Mac).

Orsak: Single-arch images - `exec format error` vid lokal test.

Lösning: Lade till multi-arch build i GitHub Actions:

```
- uses: docker/build-push-action@v6
  with:
    platforms: linux/amd64,linux/arm64
```

Lärdomar:

- Bygg alltid multi-arch för portabilitet

- Använd **docker buildx** för cross-platform builds
 - Testa images på olika arkitekturer
-

5.11 Terraform Provider Version Matrix

Problem: **terraform init** misslyckades med "no available releases match the given constraint".

Orsak: EKS module v20 kräver AWS provider v6, men VPC module v6 också kräver v6 - circular dependency.

Lösning: Downgradade till kompatibla versioner:

- EKS module: v19.0
- VPC module: v5.0
- AWS provider: ~> 5.0

Lärdomar:

- Kontrollera module compatibility matrix
 - Använd **~>** för minor version flexibility
 - Testa **terraform init** efter version changes
-

5.12 GHCR Package Permissions

Problem: EKS kunde inte pulla images från GitHub Container Registry - **ImagePullBackOff**.

Orsak: Packages var private by default.

Lösning: Ändrade package visibility till Public i GitHub settings.

Alternativ lösning för private packages:

```
imagePullSecrets:  
- name: ghcr-secret
```

Lärdomar:

- GHCR packages är private by default
 - Public packages kräver ingen authentication
 - För production: använd imagePullSecrets
-

5.13 CoreDNS Addon Degraded State

Problem: CoreDNS addon fastnade i DEGRADED state efter EKS creation.

Orsak: Addons behöver tid att starta, särskilt på spot instances.

Lösning: Ökade Terraform timeouts och väntade 5-10 minuter.

Lärdomar:

- EKS addons kan ta tid att bli Active
 - Spot instances kan fördröja startup
 - Använd `kubectl get pods -n kube-system` för att verifiera
-

Sammanfattning av Lärdomar**Planering:**

- Testa lokalt innan deploy till cloud
- Använd multi-arch builds från början
- Dokumentera alla manuella steg

Terraform:

- Validera syntax: `terraform validate`
- Kontrollera module compatibility
- Använd consistent versioning (~> 5.0)

Kubernetes:

- Planera för rolling update overhead (2x pods)
- Minst 2 nodes för multi-node testing
- Explicit security group rules för inter-node traffic

Debugging:

- `kubectl describe pod` för pod issues
- `kubectl logs` för application errors
- `kubectl get events` för cluster events
- AWS Console för infrastructure issues

Kostnad:

- Använd spot instances (70% billigare)
- Minimal node count (2 för HA)
- Destroy när inte i bruk
- Övervaka med Cost Explorer

Referenser

- Kubernetes Docs – <https://kubernetes.io/docs/>
- AWS EKS – <https://docs.aws.amazon.com/eks/>
- Terraform AWS Modules – <https://github.com/terraform-aws-modules>
- GitHub Actions Docker – <https://github.com/docker/build-push-action>